

Transaction Processing: Two-Phase Locking and Concurrency Control

21.1 Two-Phase Locking Techniques for Concurrency Control

Concurrency control ensures that multiple transactions running in parallel do not interfere with each other. A primary technique is **locking**, where each item has an associated lock state controlled by the DBMS.

21.1.1 Types of Locks and System Lock Tables

A **lock** is a variable stored in a **lock table** that indicates whether a data item is available for reading or writing.

Two main lock types:

- **Binary Locks**
- **Shared/Exclusive (Read/Write) Locks**

Binary Locks

A binary lock has only two states:

$$\text{LOCK}(X) \in \{0, 1\}$$

$$0 = \text{unlocked}, \quad 1 = \text{locked}$$

A transaction must perform:

lock_item(X) before accessing X
unlock_item(X) after it finishes

Binary Locking Rules

1. A transaction must lock an item before reading/writing it.
2. It must unlock the item when done.
3. It cannot lock an item it already holds.
4. It cannot unlock an item it does not hold.

Binary Locking Algorithms (Figure 21.1)

```
lock_item(X):
B:  if LOCK(X) = 0          (*item is unlocked*)
    then LOCK(X) ← 1      (*lock the item*)
    else
        begin
            wait (until LOCK(X) = 0
                and the lock manager wakes up the transaction);
            go to B
        end;
unlock_item(X):
    LOCK(X) ← 0;          (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

Figure 21.1
Lock and unlock operations
for binary locks.

Figure 1: Binary Lock and Unlock operations.

Implementation Notes

A lock table entry typically looks like:

⟨Data_item, LOCK, Locking_transaction, Waiting_queue⟩

Shared/Exclusive (Read/Write) Locks

Binary locks are too restrictive. Shared/Exclusive locks allow:

- Multiple readers
- Only one writer

Lock States

$LOCK(X) \in \{\text{unlocked, read-locked, write-locked}\}$

- **read-locked (shared):** Many transactions may read.
- **write-locked (exclusive):** Only one transaction may write.

Lock Table Fields

⟨Item, LOCK, No_of_reads, Locking_transactions⟩

Shared/Exclusive Locking Algorithms (Figure 21.2)

```

read_lock(X):
B:  if LOCK(X) = "unlocked"
      then begin LOCK(X) ← "read-locked";
           no_of_reads(X) ← 1
           end
      else if LOCK(X) = "read-locked"
           then no_of_reads(X) ← no_of_reads(X) + 1
      else begin
           wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;
write_lock(X):
B:  if LOCK(X) = "unlocked"
      then LOCK(X) ← "write-locked"
      else begin
           wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;
unlock (X):
      if LOCK(X) = "write-locked"
          then begin LOCK(X) ← "unlocked";
               wakeup one of the waiting transactions, if any
               end
      else if LOCK(X) = "read-locked"
          then begin
               no_of_reads(X) ← no_of_reads(X) - 1;
               if no_of_reads(X) = 0
                   then begin LOCK(X) = "unlocked";
                        wakeup one of the waiting transactions, if any
                        end
               end;

```

Figure 21.2
Locking and unlocking
operations for two-
mode (read/write, or
shared/exclusive)
locks.

Figure 2: Shared Lock and Unlock operations.

Rules for Using the Locks

1. Must issue `read_lock(X)` before `read_item(X)`.
2. Must issue `write_lock(X)` before `write_item(X)`.
3. Must issue `unlock(X)` when finished.
4. Cannot read-lock if already holding another lock.
5. Cannot write-lock if already holding a lock.
6. Cannot unlock without holding a lock.

Why Shared/Exclusive Locks Are Better

- Multiple readers allowed → higher concurrency

- Writers get exclusive control → correctness preserved
- Foundation of **Two-Phase Locking (2PL)**

Lock Conversion in Two-Phase Locking

In basic Shared/Exclusive locking, a transaction may sometimes need to **change the type of lock it holds**. This is known as **lock conversion**. Lock conversion allows a transaction to safely move between *shared (read)* and *exclusive (write)* lock states, depending on the operations it needs to perform.

Upgrading

Upgrading occurs when a transaction currently holds a **shared (read) lock** on a data item and decides that it now needs to **write** that item.

- The transaction first issued a `read_lock(X)`.
- Later, it issues a `write_lock(X)` on the same item.

This is allowed only if:

- No other transaction is currently holding a shared lock on that item.

If other transactions are reading the same item, the upgrading transaction must wait. Upgrading increases exclusiveness, and therefore may temporarily block concurrency.

Downgrading

Downgrading occurs when a transaction holds a **write (exclusive) lock** on a data item and later decides that it only needs to **read** that item.

- The transaction first issued a `write_lock(X)`.
- Later, it issues a `read_lock(X)` on the same item.

When downgrading:

- The write lock is released.
- The lock becomes a shared (read) lock.
- Other transactions may now also obtain shared locks on the item.

Downgrading increases concurrency because it relaxes exclusivity after writing is finished.

Summary

- **Upgrading:** Shared → Exclusive (reduces concurrency; may require waiting)
- **Downgrading:** Exclusive → Shared (increases concurrency; allows others to read)

Lock conversion is allowed only during the **growing phase** of Two-Phase Locking (2PL).

21.1.2 Guaranteeing Serializability by Two-Phase Locking

Two-phase locking (2PL) is a protocol that ensures serializability of schedules by restricting the order in which transactions acquire and release locks.

Definition of Two-Phase Locking

A transaction is said to obey the **two-phase locking protocol** if:

All lock acquisition operations (read_lock, write_lock) occur before the first unlock operation in the transaction.

Thus, every transaction has two phases:

1. **Growing (Expanding) Phase:** The transaction may acquire new locks, but may not release any.
2. **Shrinking Phase:** The transaction may release locks, but may not acquire any new ones.

If lock conversion is allowed:

- **Upgrading** (read $\rightarrow$ write) must occur in the growing phase.
- **Downgrading** (write $\rightarrow$ read) must occur in the shrinking phase.

Transactions That Violate 2PL

Explanation of Figure 21.3 :

- **Part (a) - The Violation:** In T_1 , a `write_lock(X)` occurs after an `unlock(Y)`. In T_2 , a `write_lock(Y)` occurs after an `unlock(X)`. These early unlocks allow the other transaction to modify data concurrently, leading to a race condition.
- **Part (b) - Correct Serial Outcomes:** If the transactions were executed serially:
 - T_1 then T_2 : $X = 50, Y = 80$
 - T_2 then T_1 : $X = 70, Y = 50$

Any valid concurrent schedule must match one of these outcomes.

- **Part (c) - Nonserializable Result:** Because 2PL was violated, both transactions read old values ($X = 20, Y = 30$) before either updated anything. T_1 computes based on old Y , and T_2 computes based on old X . Result: $X = 50, Y = 50$, which matches neither serial execution. This schedule is **nonserializable**.

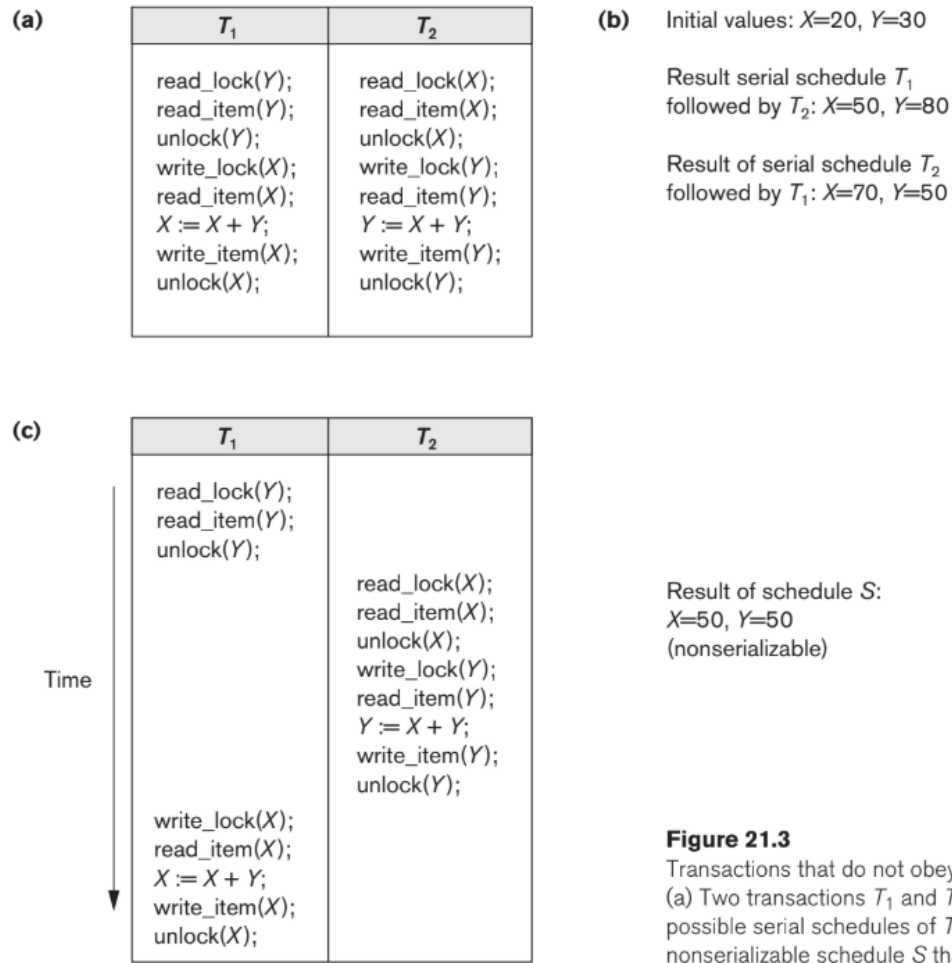


Figure 21.3

Transactions that do not obey two-phase locking. (a) Two transactions T_1 and T_2 . (b) Results of possible serial schedules of T_1 and T_2 . (c) A nonserializable schedule S that uses locks.

Figure 3: Illustration of why locking alone is not sufficient to guarantee serializability. Even though T_1 and T_2 use locks, incorrect interleaving produces a nonserializable schedule. Parts (a), (b), and (c) are shown in the figure.

Figure 21.4

Transactions T_1' and T_2' , which are the same as T_1 and T_2 in Figure 21.3 but follow the two-phase locking protocol. Note that they can produce a deadlock.

T_1'	T_2'
read_lock(Y); read_item(Y); write_lock(X); unlock(Y) read_item(X); $X := X + Y$; write_item(X); unlock(X);	read_lock(X); read_item(X); write_lock(Y); unlock(X) read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);

Figure 4: Figure 21.4

Enforcing Two-Phase Locking

If 2PL is enforced, transactions must be rewritten so that all locks are obtained before any unlock occurs. Figure 21.4 shows corrected transactions T_1' and T_2' that

follow the 2PL protocol.

Under 2PL:

- T'_1 will request `write_lock(X)` before releasing Y .
- Therefore, when T'_2 requests `read_lock(X)`, it must wait.

However, this waiting may cause **deadlock**, a disadvantage of 2PL.

Serializability Guarantee

A key theoretical result is:

If every transaction in a schedule follows the Two-Phase Locking protocol, the schedule is guaranteed to be serializable.

This removes the need to test schedules for serializability.

Drawbacks of Two-Phase Locking

Although 2PL guarantees serializability, it does **not** allow all serializable schedules. It may reduce concurrency because:

- A transaction cannot release an item early if it still needs to lock another item later.
- A transaction may have to lock an item earlier than necessary just to follow 2PL rules.

This causes other transactions to wait unnecessarily.

Variations of Two-Phase Locking (2PL)

Two-Phase Locking (2PL) is a concurrency control technique that ensures conflict serializability by dividing a transaction's lock behavior into two phases: a *growing phase*, where locks may be acquired but not released, and a *shrinking phase*, where locks may be released but not acquired. Several variants of 2PL exist, each with its own advantages and drawbacks.

1. Basic 2PL

Definition: In Basic 2PL, a transaction may obtain locks at any time during its execution. However, once it releases *any* lock, it cannot acquire additional locks. The transaction therefore has a growing phase followed by a shrinking phase.

Properties:

- Guarantees conflict serializability.
- Does **not** prevent deadlocks.

- Locks may be released as soon as the transaction no longer needs them.

Usage: Implemented in many systems, though often enhanced for recovery safety.

2. Conservative (Static) 2PL

Definition: Conservative 2PL requires a transaction to obtain *all* the locks it will ever need *before* the transaction begins execution. If any lock is not available, the transaction waits.

Mechanism:

- Transaction must predeclare its entire read-set and write-set.
- Execution begins only when all requested locks are granted.

Advantages:

- **Deadlock-free:** since no transaction requests locks while holding others.
- Ensures serializability.

Disadvantages:

- Requires advance knowledge of all data items to be accessed.
- Difficult to use for dynamic queries, loops, or interactive inputs.

Usage: Rarely used in practice; mostly theoretical or for highly predictable workloads.

3. Strict 2PL

Definition: Strict 2PL is a stronger variant in which a transaction holds all *exclusive locks* (write locks) until after it commits or aborts. Shared (read) locks may be released earlier depending on the system.

Advantages:

- Ensures **strict schedules**.
- Prevents dirty reads and dirty writes.
- Simplifies recovery and rollback since uncommitted data is never read.

Disadvantages:

- **Not** deadlock-free.

Usage: Most modern DBMS use Strict 2PL due to its strong correctness and recovery benefits.

Variant	Serializable	Deadlock-Free	Write Locks Held Until Commit
Basic 2PL	Yes	No	No
Conservative 2PL	Yes	Yes	Not Required
Strict 2PL	Yes	No	Yes

Summary Table

Deadlock and Starvation in Concurrency Control

Deadlock

A **deadlock** occurs in a database system when a set of two or more transactions are waiting for one another in a circular chain. Each transaction in the set is waiting for a lock that another transaction in the cycle holds, and thus none of them can proceed.

Formally, deadlock occurs if there exists a set of transactions $\{T_1, T_2, \dots, T_n\}$ such that:

T_1 is waiting for T_2 , T_2 is waiting for T_3 , $\dots$, T_n is waiting for T_1 .

Because each transaction is blocked waiting for another transaction in the cycle, none of them can release their locks, resulting in an infinite wait.

A simple example is illustrated in Figure ???. Transaction T_1' locks item Y and requests item X , while transaction T_2' locks item X and requests item Y . Neither can proceed.

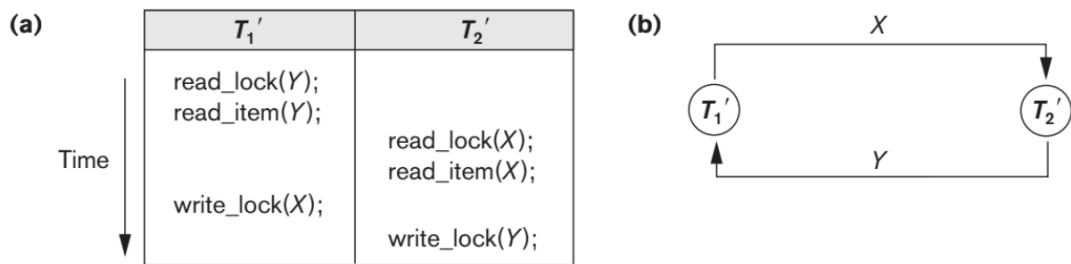


Figure 21.5

Illustrating the deadlock problem. (a) A partial schedule of T_1' and T_2' that is in a state of deadlock. (b) A wait-for graph for the partial schedule in (a).

Figure 5: Figure showing deadlock

Deadlock Prevention Protocols

Deadlock prevention techniques ensure that the system never enters a deadlock state. However, prevention often reduces concurrency and may require additional overhead.

There are several prevention strategies:

1. Conservative (Static) Two-Phase Locking

Also called **static 2PL**, this protocol requires that a transaction lock **all the items it will ever need** before it begins execution.

- If any lock cannot be obtained, the transaction releases all locks and retries later.
- Deadlock cannot occur because a transaction never waits after acquiring some locks.
- Impractical because most transactions do not know their read-set and write-set in advance.
- Dramatically reduces concurrency.

2. Lock Ordering (Graph-Based Protocol)

Every data item is assigned a unique, global order (e.g., alphabetical or numeric). Transactions must acquire locks **only in this fixed order**.

- Eliminates cycles in the wait-for graph.
- Guarantees deadlock-free execution.
- Requires advance knowledge of item order.
- Difficult in dynamic databases.

Timestamp-Based Deadlock Prevention

More practical deadlock-prevention algorithms use the concept of **transaction timestamps**. Each transaction T is assigned a unique timestamp $TS(T)$ when it begins.

If T_i starts before T_j , then $TS(T_i) < TS(T_j)$

Older transactions have smaller timestamps.

These prevention schemes decide whether a transaction should wait or abort based on the comparison of timestamps.

Two well-known schemes are:

Wait–Die Scheme

Suppose transaction T_i requests a lock on item X , but X is already locked by T_j .

If $TS(T_i) < TS(T_j) \Rightarrow T_i$ waits

Else $TS(T_i) > TS(T_j) \Rightarrow T_i$ aborts (dies) and restarts

Properties:

- Non-preemptive: Young transactions abort themselves.
- Old transactions are allowed to wait.
- Prevents deadlock because only older transactions wait.

Wound–Wait Scheme

Suppose transaction T_i wants an item locked by T_j :

If $TS(T_i) < TS(T_j) \Rightarrow T_j$ is aborted (wounded)

Else $TS(T_i) > TS(T_j) \Rightarrow T_i$ waits

Properties:

- Preemptive: A transaction may force another to abort.
- Older transactions never wait for younger ones.
- Younger transactions must wait or abort.

Starvation

Starvation occurs when a transaction keeps waiting indefinitely and never gets the resources it needs. Starvation can arise in systems where some transactions are repeatedly restarted or always bypassed.

Reasons for starvation:

- A transaction is repeatedly aborted (e.g., always the “younger” one).
- Lock ordering causes delays.
- High-priority or older transactions continually preempt younger ones.

How Timestamp Protocols Avoid Starvation

In both wait–die and wound–wait:

- A transaction **keeps its original timestamp even after restart**.
- Eventually, it becomes one of the oldest transactions.
- Thus, it will no longer be aborted.

Therefore, timestamp-based schemes prevent both:

- Deadlock, by restricting waiting behavior.
- Starvation, by ensuring restarted transactions eventually become old enough.

Concurrency Control Based on Timestamp Ordering

The use of locking together with the two-phase locking (2PL) protocol ensures serializability by enforcing an order based on the time a transaction acquires its locks. However, 2PL may cause blocking and deadlock, and aborted transactions may need to restart.

An alternative to locking is the use of **timestamps**, which orders transaction operations based on the transaction start time. This eliminates deadlocks entirely. We discuss timestamps in Section and the timestamp ordering algorithm in Section .

Timestamps

A **timestamp** is a unique identifier assigned to each transaction by the DBMS. It typically reflects the order in which transactions enter the system, so:

$$TS(T_i) < TS(T_j) \quad \text{if transaction } T_i \text{ starts before } T_j.$$

Timestamp-based concurrency control does **not use locks**, so deadlocks cannot occur.

There are two common methods for generating timestamps:

1. **Monotonic counter:** Each new transaction receives the next integer in the sequence $1, 2, 3, \dots$. The counter is reset only when no transactions are active.
2. **System clock:** Use the current date/time value, ensuring no two timestamps are generated in the same clock tick.

The Timestamp Ordering Algorithm

The idea of timestamp ordering (TO) is to ensure that all conflicting operations across transactions occur in **timestamp order**. This guarantees conflict serializability.

Unlike 2PL—where a schedule may be equivalent to *some* serial order—timestamp ordering enforces serializability according to the **specific order of timestamps**.

To enforce this, every database item X is associated with two timestamp values:

1. **read_TS(X):** The largest timestamp among all transactions that have successfully read X .

$$read_TS(X) = \max\{TS(T) \mid T \text{ read } X\}$$

2. **write_TS(X):** The largest timestamp among all transactions that have successfully written X .

$$write_TS(X) = \max\{TS(T) \mid T \text{ wrote } X\}$$

Basic Timestamp Ordering Algorithm

When a transaction T issues a `read_item(X)` or `write_item(X)` operation, the algorithm compares $TS(T)$ with the timestamps of X . If the timestamp order is violated, the operation is rejected and T is **aborted and restarted** with a new timestamp.

This may cause **cascading rollback**, because any transaction that used a value written by T must also be rolled back.

The algorithm checks two cases:

Case 1: Transaction issues `write_item(X)`

(a) If

$$read_TS(X) > TS(T) \quad \text{or} \quad write_TS(X) > TS(T)$$

then abort T which is your older transaction. This means a younger transaction with a timestamp greater than the timestamp of T had already read or wrote X , violating timestamp order.

(b) Otherwise, perform the write and set

$$write_TS(X) = TS(T).$$

Case 2: Transaction issues `read_item(X)`

(a) If

$$write_TS(X) > TS(T),$$

then abort T . A younger transaction has already written X , so reading it would violate timestamp order.

(b) Otherwise, allow the read and update

$$read_TS(X) = \max\{read_TS(X), TS(T)\}.$$

Basic TO always aborts the later transaction in any timestamp-order violation. Thus, all schedules produced by TO are conflict serializable.

However, TO does not prevent **starvation**—a transaction may be aborted repeatedly.

Strict Timestamp Ordering

Strict TO ensures that all schedules are:

- strict,
- conflict serializable, and
- free of cascading rollback.

If a transaction T wants to read or write an item X whose current value was written by transaction T' , and

$$TS(T) > write_TS(X) = TS(T'),$$

then T must wait until T' has committed or aborted.

This simulates holding a *temporary lock* on X for recoverability, but does not cause deadlock because waiting is restricted:

A transaction waits only for older transactions (smaller timestamps).

Thomas's Write Rule

Thomas's write rule modifies basic TO to increase concurrency by allowing some out-of-order writes to be ignored.

For a transaction T performing `write_item(X)`:

1. If

$$read_TS(X) > TS(T),$$

then abort T . (Same reason as basic TO.)

2. If

$$write_TS(X) > TS(T),$$

then **ignore** the write. A younger transaction already wrote X , so the older write is **obsolete**.

3. Otherwise, perform the write and update

$$write_TS(X) = TS(T).$$

Thomas's write rule does not guarantee conflict serializability, but it **increases concurrency** by allowing outdated writes to be skipped.

Real Database Examples for Timestamp Ordering

1 Basic Timestamp Ordering (TO)

Scenario: Banking Database — Account Balances

Database item: `Account.Balance`

Version	Value	read_TS	write_TS
Balance_0	1000	0	1

Table 1: Initial version of Account Balance

Transactions:

- T_1 : starts at $TS = 5$, reads Balance.
- T_2 : starts at $TS = 10$, writes Balance = 1200.
- T_3 : starts at $TS = 4$, writes Balance = 900.

Step-by-step Execution:

1. T_1 reads Balance: $write_TS(Balance_0) = 1 < TS(T_1) = 5$, **allowed**. Update: $read_TS(Balance_0) = 5$.
2. T_2 writes Balance = 1200: check $read_TS(Balance_0) = 5 < TS(T_2) = 10$, **allowed**. Create new version: Balance_1 = 1200, $write_TS = 10$.
3. T_3 writes Balance = 900: $read_TS(Balance_0) = 5 > TS(T_3) = 4$, **abort**. Reason: Older transaction cannot overwrite a value read by newer transaction.

Version	Value	read_TS	write_TS
Balance_0	1000	5	1
Balance_1	1200	0	10

Table 2: State of database after Basic TO operations

2 Strict Timestamp Ordering (Strict TO)

Goal: Avoid cascading rollback.

Scenario: Banking Database — Account Balances

- T_1 : starts at $TS = 5$, writes Balance = 1100 (uncommitted).
- T_2 : starts at $TS = 10$, reads Balance.

Step-by-step Execution:

1. T_1 writes Balance = 1100; temporary version Balance_1 created.
2. T_2 wants to read Balance: $write_TS(Balance_1) = 5$ (uncommitted), **wait**. T_2 waits until T_1 commits.
3. T_1 commits: Balance_1 = 1100 becomes official.
4. T_2 reads Balance = 1100. No cascading rollback occurs.

Version	Value	read_TS	write_TS
Balance_0	1000	0	1
Balance_1	1100	10	5

Table 3: State of database after Strict TO operations

3 Thomas's Write Rule (TWR)

Goal: Increase concurrency by ignoring obsolete writes.

Scenario: Banking Database — Account Balances

- T_1 : $TS = 5$, writes Balance = 1100.
- T_2 : $TS = 10$, writes Balance = 1200.
- T_1 (late write): attempts Balance = 1150.

Step-by-step Execution:

1. T_1 writes Balance = 1100: allowed, creates Balance_1, $write_TS = 5$.
2. T_2 writes Balance = 1200: allowed, creates Balance_2, $write_TS = 10$.
3. T_1 late write = 1150: $write_TS(Balance_2) = 10 > TS(T_1) = 5$, **ignored**.
Reason: older write is obsolete; do not abort.

Version	Value	read_TS	write_TS
Balance_0	1000	0	1
Balance_1	1100	0	5
Balance_2	1200	0	10

Table 4: State of database after Thomas Write Rule applied

Summary of Examples

- **Basic TO:** aborts any transaction violating timestamp order.
- **Strict TO:** waits for uncommitted older transactions; prevents cascading roll-back.
- **Thomas Write Rule:** ignores obsolete writes; allows more concurrency without aborting.

Key Takeaways: Timestamp ordering is crucial for serializability in high-concurrency database systems. Strict TO is safer for recovery. Thomas Write Rule is optimized for performance.

Multiversion Concurrency Control (MVCC) Detailed Examples

1. Overview of MVCC

Traditional single-version databases store **one value per item**, e.g., $X = 10$.

Problem: If a transaction reads X while another writes it, either:

- The reader sees an inconsistent value, or
- The writer must wait, creating **blocking**.

MVCC Solution: Maintain **multiple versions** of each data item:

$$X_0, X_1, X_2, \dots$$

- Readers can read older versions without waiting.
- Writers create a new version for each update.
- Concurrency increases while consistency is preserved.

2. MVCC Based on Timestamp Ordering (MVTO)

Key concepts:

- Each transaction T receives a unique **start timestamp** $ts(T)$.
- Each version X_i of a data item maintains:
 - * $write_TS(X_i)$: timestamp of transaction that created it.
 - * $read_TS(X_i)$: largest timestamp of transactions that read it.

Rules for reads and writes:

1. **Read by T :** Read the newest version X_i with $write_TS(X_i) \leq ts(T)$; update $read_TS(X_i) := \max(read_TS(X_i), ts(T))$.
2. **Write by T :** If any version X_j has $read_TS(X_j) > ts(T)$, abort T ; otherwise, create new version X_{new} with $write_TS(X_{new}) = ts(T)$.

3. Step-by-Step Example — Simple MVTO

Initial state:

Version	Value	read_TS	write_TS
X_0	10	0	1

Table 5: Starting version of data item X

Transactions:

- T_1 : $ts(T_1) = 5$ (reads)
- T_2 : $ts(T_2) = 10$ (writes $X = 20$)

Actions:

1. T_1 reads X : newest version with $write_TS \leq 5 \rightarrow X_0$. Update $read_TS(X_0) = 5$.

2. T_2 writes $X = 20$: $read_TS(X_0) = 5 \leq ts(T_2) = 10$, so allowed. Create $X_1 = 20$, $write_TS(X_1) = 10$.
3. T_1 reads X again: only versions with $write_TS \leq 5 \rightarrow$ reads $X_0 = 10$.

Version Table After Transactions:

Version	Value	read_TS	write_TS
X_0	10	5	1
X_1	20	0	10

Table 6: MVTO version table after reads and writes

4. Example — Write Rejection

Suppose T_3 starts with $ts(T_3) = 4$ and tries to write X .

Check X_0 : $read_TS(X_0) = 5 > ts(T_3) = 4 \rightarrow$ **cannot write**.

Reason: T_3 is too old — a newer transaction T_1 already read X_0 .

Lesson: In MVTO, older transactions cannot overwrite versions used by newer ones.

5. Multiversion 2-Phase Locking (MV2PL)

MV2PL combines **locks** with **multiple versions**.

Lock Types:

- Read (R) — multiple readers allowed.
- Write (W) — exclusive for preparing a new version.
- Certify (C) — final commit-phase lock.

Typical flow:

1. Acquire W-lock, make private copy.
2. Request C-lock to commit; new version becomes visible.
3. Release locks; readers can access updated version.

Lock Compatibility Table:

	R	W	C
R	Yes	No	No
W	No	No	No
C	No	No	No

Table 7: MV2PL Lock Compatibility

6. MV2PL Worked Example

Initial state: $X_0 = 50$

Transactions: T_1 and T_2

1. T_1 wants to update X : acquires W-lock, creates private $X_{priv} = 60$.
2. T_2 wants to read X : blocked until T_1 commits.
3. T_1 requests C-lock $\rightarrow$ granted $\rightarrow X_1 = 60$ visible.
4. T_2 now reads $X_1 = 60$.

7. Comparison: MVTO vs MV2PL

Feature	MVTO (Timestamp)	MV2PL (Locks)
Basis	Timestamps	Locks (R, W, C)
Readers block writers?	No	Sometimes
Writers block readers?	No	Yes (during W-lock)
Uses old versions?	Yes	Yes
Certify/commit step?	No	Yes
Concurrency	High	Moderate

Table 8: Comparison of MVTO and MV2PL

8. Practical Notes

- Widely used in PostgreSQL, Oracle, and InnoDB.
- Old versions must be garbage collected (vacuum) to reclaim space.
- MVTO maximizes concurrency; MV2PL is easier to reason about for serializability.
- Timestamp schemes must handle clock skew or monotonic counters.

9. Combined Timeline Example

— MVTO

Initial: $X_0.value = 10$, $write_TS(X_0)=1$
 $T_1(ts=5)$ reads $X \rightarrow$ reads $X_0=10$, $read_TS(X_0)=5$
 $T_2(ts=10)$ writes $X=20 \rightarrow$ creates X_1 , $write_TS(X_1)=10$
 T_1 reads X again $\rightarrow$ reads $X_0=10$ (old version)

— MV2PL

T_1 acquires W-lock on X , prepares $X_{priv}=60$
 T_2 requests R-lock $\rightarrow$ blocked
 T_1 gets C-lock, commits $\rightarrow X_1=60$ visible
 T_2 unblocked $\rightarrow$ reads $X_1=60$

Optimistic Techniques, Snapshot Isolation, and Multiple Granularity Locking

Optimistic (Validation) Techniques

Overview: Optimistic techniques, also called *validation* or *certification* methods, assume that conflicts between transactions are rare. Therefore, transactions execute without acquiring locks and are validated at the end before committing.

This is called “optimistic” because the system *optimistically assumes* that conflicts are unlikely, unlike traditional locking which prevents conflicts upfront.

Key Points:

- Transactions execute **without locks** on data items.
- Updates are stored in **local copies** of data items.
- During the **validation phase**, the system checks if committing the transaction would violate serializability (correct order of transactions).
- If a conflict is detected, the transaction is aborted; otherwise, changes are applied to the database.

Real Database Example: Banking System

- Database items: `Account.Balance` and `Account.Limit`.
- Transaction T_1 : transfer \$200 from Account A to Account B.
- Transaction T_2 : apply interest on Account A (5%).

Execution Step-by-Step:

1. Both T_1 and T_2 start with local copies of Account A.
2. T_1 reads Account A = 1000, Account B = 500; subtracts 200 → new local balances: A=800, B=700.
3. T_2 reads Account A = 1000; adds 5% interest → new local balance: A=1050.
4. **Validation phase:** The system detects that both transactions updated Account A based on the same original value.
5. Conflict resolution: T_1 commits first; T_2 is aborted to preserve consistency.
6. Database after commit: Account A = 800, Account B = 700.

Transaction	Account A	Account B	Status
T1 (transfer)	800	700	committed
T2 (interest)	1000	-	aborted

Table 9: Optimistic validation example showing conflict resolution

Why it works: Optimistic validation avoids the overhead of locks, improving concurrency. However, transactions may be aborted if conflicts occur, which can lead to wasted work if conflicts are frequent.

Snapshot Isolation (SI)

Overview: Snapshot Isolation allows each transaction to see a **consistent snapshot** of the database as of the start time of the transaction. This avoids read-write blocking and provides better concurrency than traditional locking.

Key Points:

- Reads see only committed values at the transaction start (snapshot).
- Read operations do not require locks (non-blocking reads).
- Writes require locks to create new versions (temporary storage for updated values).
- Variation: *Serializable Snapshot Isolation (SSI)* guarantees full serializability.

Real Database Example: Online Store Inventory

- Database items: `Product.Stock`.
- Transaction T_1 : reads stock of Product P to decide purchase.
- Transaction T_2 : sells 10 units of Product P.

Execution Step-by-Step:

1. T_1 starts → sees snapshot of Stock = 50 units.
2. T_2 sells 10 units → new version created → Stock = 40.
3. T_1 reads Stock → still sees 50 (snapshot does not include T_2 's changes yet).
4. T_1 decides to buy 5 units → writes succeed if no other transaction has updated the same item in the meantime.

Transaction	Snapshot Seen	Action
T1	Stock = 50	reads and writes 5 units based on snapshot
T2	Stock = 50	sells 10 units → Stock = 40 committed

Table 10: Snapshot Isolation example: transactions work on snapshots

Why it works: Snapshot isolation avoids readers blocking writers and vice versa. Each transaction works with a consistent view of the database. However, write-write conflicts are still checked during commit.

Granularity of Data Items and Multiple Granularity Locking (MGL)

Granularity: The size of the data items that are locked in a database affects concurrency:

- Fine-grained (records, rows): higher concurrency, more locks needed.
- Coarse-grained (tables, disk blocks): lower concurrency, fewer locks, less overhead.

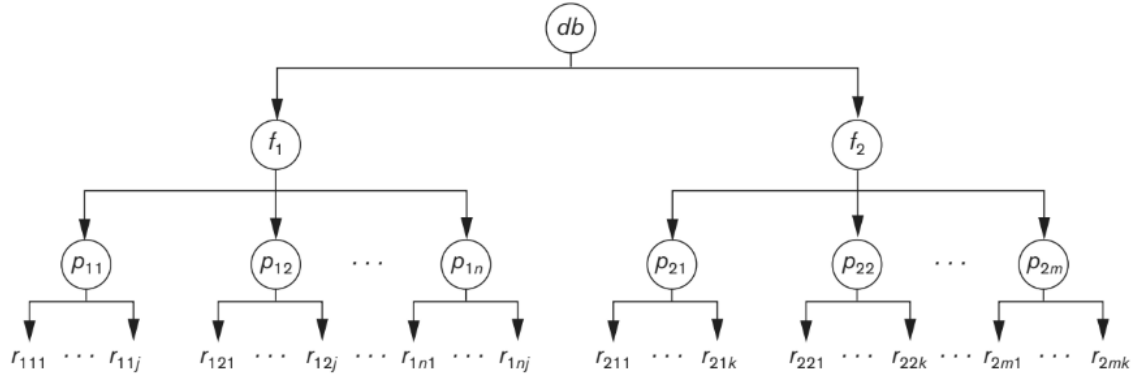


Figure 21.7

A granularity hierarchy for illustrating multiple granularity level locking.

Figure 6: Granularity hierarchy.

Beginner-Friendly Explanation of Lock Types

Before understanding intention locks, it is important to first understand the basic lock types used in databases. Here is a simple explanation:

Shared (S) Lock – “I only want to read”

A **Shared (S)** lock means:

- The transaction wants to **read** the data.
- It promises that it will NOT change the data.

Real-life example: Imagine a library. If you only want to *read* a book inside the library, others can read it at the same time. No problem.

Compatibility: Many transactions can hold an S lock on the same item at the same time.

Exclusive (X) Lock – “I want to update/change”

An **Exclusive (X)** lock means:

- The transaction wants to **write or update** the data.
- No one else is allowed to read or write the item during this time.

Real-life example: If you want to *edit* a book, you must take it away so nobody can read it while you write inside it.

Compatibility: Only ONE transaction can hold an X lock on an item.

Why S and X are not enough

If a transaction locks a **big object** (like a file or table), and another transaction locks a **small object** (like a single record), the database must check every small object under the big object for conflicts. This becomes very slow.

To fix this, we use **intention locks**.

Intention-Shared (IS) Lock – "I plan to read something inside"

An **IS lock** means:

- The transaction does NOT want to lock the node itself.
- The transaction intends to acquire **Shared (S)** locks on some lower-level items.

Example: You enter the library building because you plan to read a chapter from a book. You are not locking the whole building; you are just saying:

"I will READ something inside."

Intention-Exclusive (IX) Lock – "I plan to write something inside"

An **IX lock** means:

- The transaction intends to acquire **Exclusive (X)** locks on some lower-level items.

Example: You enter the library building because you want to write something in some book. You don't lock the entire building, but you say:

"I will UPDATE something inside."

Shared + Intention-Exclusive (SIX) Lock – "I want to read everything here, but also update some parts"

A **SIX lock** means:

- The transaction has a **Shared** lock on the node (can read entire file/table).
- It will also take **Exclusive** locks on some lower items (update specific records).

Example: You borrow a book for reading (S lock), but you also say:

"I will edit ONE paragraph inside this book."

This is why SIX = Shared + Intention Exclusive.

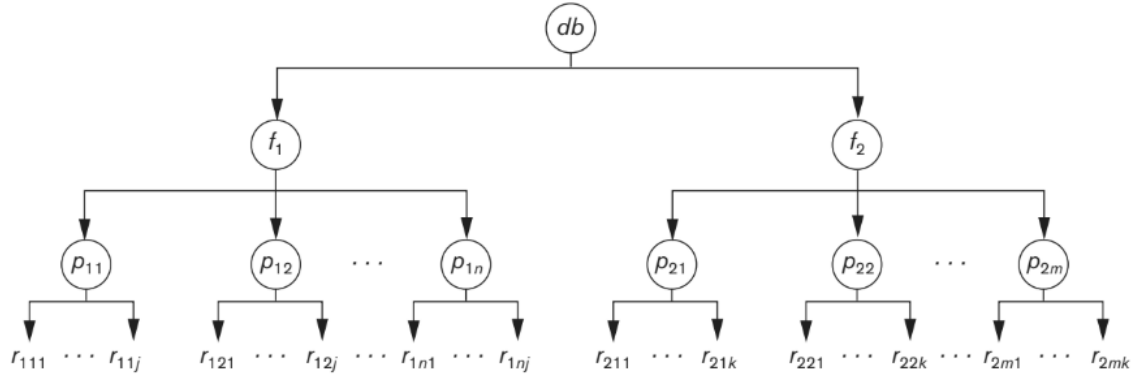


Figure 21.7

A granularity hierarchy for illustrating multiple granularity level locking.

Figure 7: Granularity Hierarchy

Summary Table (Beginner View)

- **S** = I will READ this item.
- **X** = I will WRITE/UPDATE this item.
- **IS** = I will read something *inside* this item.
- **IX** = I will write something *inside* this item.
- **SIX** = I will read everything here, and write to some parts.

Intention locks help the database quickly determine conflicts without checking every record or page. They also allow different transactions to work safely at different levels of the hierarchy (database, files, pages, records).

Multiple Granularity Level Locking (MGL)

In many database systems, data items are arranged in a hierarchy such as:

Database $\rightarrow$ File $\rightarrow$ Page $\rightarrow$ Record

This hierarchy is shown. Locking at different granularities allows transactions to lock coarse or fine data items depending on their needs.

Motivation for Multiple Granularity Locking

Consider a transaction T_1 that wants to *update all records* in file f_1 . If T_1 acquires an **exclusive (X)** lock on file f_1 , then all pages ($p_{11}, p_{12}, \dots, p_{1n}$) and all records within them are implicitly locked.

This is efficient because:

- Instead of locking n pages or hundreds of records, T_1 locks only one node (file).

Now consider another transaction T_2 that only wants to *read* a single record r_{1nj} on page p_{1n} .

Without intention locks:

- If T_2 requests a shared lock on record r_{1nj} before T_1 's file-level X-lock,
- then when T_1 later requests a file-level X-lock,
- the lock manager must check *every descendant* of f_1 for conflicting locks.

This requires checking all pages and records under f_1 , which is expensive and defeats the purpose of granularity.

Need for Intention Locks

To avoid costly checking, special locks called **intention locks** are introduced. These locks indicate the type of lock a transaction intends to acquire on lower-level nodes.

Types of Intention Locks

- **Intention-Shared (IS)**: The transaction intends to acquire shared locks on descendant nodes.
- **Intention-Exclusive (IX)**: The transaction intends to acquire exclusive locks on descendant nodes.
- **Shared with Intention-Exclusive (SIX)**: The node itself is locked in shared mode, but the transaction intends to acquire exclusive locks on some descendants.

Lock Compatibility Matrix

	IS	IX	S	SIX	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	No	No
SIX	Yes	No	No	No	No
X	No	No	No	No	No

Table 11: Compatibility of Intention, Shared, and Exclusive Locks

Multiple Granularity Locking (MGL) Protocol

The MGL protocol imposes the following rules:

1. **Lock compatibility rules must be followed** (Table above).
2. **The root of the tree must be locked first** (in any mode).

■ Multiple granularity locking (MGL) protocol rules

1. The lock compatibility (based on Figure 21.8) must be adhered to.
2. The root of the tree must be locked first, in any mode.
3. A node N can be locked by a transaction T in S or IS mode only if the parent node N is already locked by transaction T in either IS or IX mode.
4. A node N can be locked by a transaction T in X, IX, or SIX mode only if the parent of node N is already locked by transaction T in either IX or SIX mode.
5. A transaction T can lock a node only if it has not unlocked any node (to enforce the 2PL protocol).
6. A transaction T can unlock a node, N , only if none of the children of node N are currently locked by T .

Figure 8: Multiple Granularity Rules

3. A node N may be locked in **S or IS** mode only if the parent of N is already locked in **IS or IX**.
4. A node N may be locked in **X, IX, or SIX** mode only if the parent of N is already locked in **IX or SIX**.
5. **2PL must be followed:** a transaction cannot request additional locks once it has unlocked any node.
6. A transaction may **unlock a node only if none of its children are currently locked**.

These rules ensure that:

- Locking proceeds from root downward.
- Unlocking proceeds from leaf upward.
- Serializability is preserved.

T_1	T_2	T_3
IX(db) IX(f_1)	IX(db)	IS(db) IS(f_1) IS(p_{11})
IX(p_{11}) X(r_{111})	IX(f_1) X(p_{12})	S(r_{11j})
IX(f_2) IX(p_{21}) X(p_{211})		
unlock(r_{211}) unlock(p_{21}) unlock(f_2)		S(f_2)
unlock(r_{111}) unlock(p_{11}) unlock(f_1) unlock(db)	unlock(p_{12}) unlock(f_1) unlock(db)	
		unlock(r_{11j}) unlock(p_{11}) unlock(f_1) unlock(f_2) unlock(db)

Figure 21.9
Lock operations to
illustrate a serializable
schedule.

Explanation of Figure 21.9 (Beginner-Friendly)

Figure 21.9 illustrates how three transactions acquire and release locks under the Multiple Granularity Locking (MGL) protocol. The hierarchy of data items is

$$db \rightarrow f_1, f_2 \rightarrow p_{11}, p_{12}, p_{21}, \dots \rightarrow r_{111}, r_{11j}, r_{211}, \dots$$

Transaction T_1

Transaction T_1 wants to **update two individual records**: r_{111} in f_1 and r_{211} in f_2 . Because it will perform updates, it must acquire **Intention-Exclusive (IX)** locks on each ancestor and **Exclusive (X)** locks on the specific records.

- IX(db), IX(f_1), IX(p_{11}), X(r_{111})
- IX(f_2), IX(p_{21}), X(r_{211})

After finishing, it unlocks in the opposite direction (bottom-up): first the records, then the pages, the files, and finally the database.

Transaction T_2

Transaction T_2 updates **all records on page** p_{12} . Since it will perform a write, it locks ancestors in IX mode and the page itself in X mode:

- IX(db), IX(f_1), X(p_{12})

It then unlocks in bottom-up order.

Transaction T_3

Transaction T_3 is a **reader**. It wants to read a single record r_{11j} and then read the entire file f_2 .

For reading a record, it acquires IS locks on ancestors and an S lock on the record. To read an entire file, it acquires an S lock at the file level.

- IS(db), IS(f_1), IS(p_{11}), S(r_{11j})
- S(f_2)

Unlocking again occurs bottom-up.

Key Points Demonstrated

- All locking is performed **top-down** (from db $\rightarrow$ file $\rightarrow$ page $\rightarrow$ record).
- Unlocking is done **bottom-up**.
- Intention locks (IS and IX) allow the lock manager to efficiently detect conflicts.
- The resulting schedule is conflict-serializable and obeys the MGL protocol.